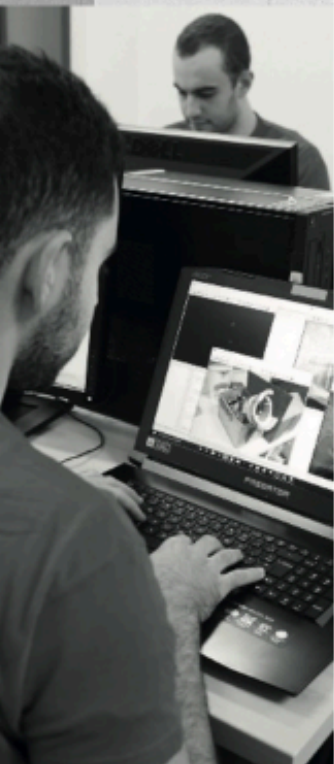




ENSEA

Beyond Engineering

COMPTE-RENDU PROJET 3A

Supervision Active & Auto-remédiation Réseau

M. BARÈS

Tristan BRAU
Léo PHAV

Sommaire

1. Introduction

- 1.1. Contexte et Enjeux
- 1.2. Problématique
- 1.3. Objectifs Techniques
- 1.4. But Final

2. Architecture de la Solution

- 2.1. Choix du service de Supervision
- 2.2. Schéma Global du Flux de Données
- 2.3. Mise en œuvre du Docker-Compose

3. Collecte et Stockage des Métriques

- 3.1. Architecture de collecte : Le modèle "Pull"
- 3.2. Supervision distante via SNMP Exporter
- 3.3. Node Exporter : Métriques de la machine locale

4. Visualisation et Alerting

- 4.1. Grafana : Dashboarding et Observabilité
- 4.2. Alertmanager : La logique de détection
- 4.3. Notification des alertes

5. Sécurité et Automatisation : Le module Python-Blocker

- 5.1. Architecture du module : Flask comme centre de commande
- 5.2. Sécurisation des échanges
- 5.3. Protocole de Test et Résultats

6. Guide de Déploiement

- 6.1. Prérequis et Lancement Initial
- 6.2. Vérification de l'état des conteneurs
- 6.3. Monitoring en temps réel des actions de défense

7. Conclusion

1. Introduction

Dans le cadre de notre dernière année de cycle ingénieur à l'ENSEA, nous avons réalisé un projet de fin d'études portant sur la supervision avancée et la sécurisation proactive d'infrastructures réseaux. Ce projet nous a été confié par M. BARÈS et a été mené en collaboration par Tristan BRAU et Léo PHAV.

1.1. Contexte et Enjeux

La complexité croissante des architectures réseaux modernes exige des outils de monitoring capables de traiter un volume massif de données en temps réel. Pour les infrastructures de l'ENSEA, l'enjeu est double : assurer une visibilité totale sur l'état des équipements via le protocole SNMP et garantir une continuité de service face à des comportements réseaux atypiques ou malveillants.

1.2. Problématique

Comment concevoir une architecture logicielle intégrée qui dépasse le stade de la simple observation pour devenir un système de défense actif ?

1.3. Objectifs Techniques

Notre solution repose sur une architecture micro-services conteneurisée via Docker-Compose, articulée autour de quatre axes majeurs :

1. Observabilité multi-sources : Déploiement de SNMP Exporter pour les serveurs de l'école et de Node Exporter pour la supervision des machines locales.
2. Gestion de la donnée : Centralisation et requêtage des métriques temporelles via Prometheus.
3. Intelligence et Alerting : Configuration d'Alertmanager pour l'identification de seuils critiques et de patterns d'attaque (haut trafic).
4. Remédiation Automatisée : Implémentation d'un service Python-Blocker, agissant comme un IPS (Intrusion Prevention System), capable de piloter dynamiquement les règles de filtrage (ports) en cas d'alerte.

1.4. But Final

L'objectif de ce projet est de démontrer notre capacité à concevoir une solution d'ingénierie logicielle complète, alliant administration système, réseaux et cybersécurité. Ce rapport détaille la conception technique, les choix d'architecture et les tests de validation.

2. Architecture de la Solution

Pour répondre aux exigences de portabilité et de rapidité de déploiement, nous avons opté pour une architecture micro-services pilotée par Docker-Compose. Cette approche permet d'isoler chaque composant tout en facilitant leur communication via un réseau virtuel interne.

2.1. Choix du service de Supervision

Pour ce projet, nous avons comparé plusieurs solutions de supervision du marché afin de sélectionner celle la plus adaptée à nos besoins de portabilité, de performance et d'auto-remédiation. Le tableau comparatif ci-dessous détaille les critères ayant conduit au choix du couple **Prometheus + Grafana**.

Critères	Nagios	Zabbix	LibreNMS	Prometheus + Grafana
Architecture	Monolithique	Monolithique	Spécialisée Réseau	Micro-services
Modèle de Collecte	Push / Check actif	Push / Agent / SNMP	Pull (SNMP Discovery)	Pull (Scraping)
Stockage	Base SQL	Base SQL	MariaDB / MySQL	TSDB (Haute Performance)
Dashboarding	Très daté	Correct (natif)	Spécifique Réseau	Grafana (Standard du marché)
Auto-remédiation	Via plugins complexes	Scripts internes	Alertes SNMP basiques	Webhooks Natifs & API Flask
Facilité Docker	Faible	Moyenne	Moyenne	Excellente (Natif)

Le choix s'est porté sur **Prometheus et Grafana** pour plusieurs raisons :

- Architecture Micro-services : Parfaitement adaptée à un déploiement via Docker-Compose.
- Modèle Pull : Permet de contrôler le flux de données et d'éviter la submersion du système de supervision.
- Performance : L'utilisation d'une base de données temporelle (TSDB) assure une gestion optimale des métriques massives en temps réel.
- Extensibilité : La facilité d'intégration de Webhooks natifs permet de piloter notre module de remédiation personnalisé (Python-Blocker) via une API Flask.

2.2. Schéma Global du Flux de Données

Le fonctionnement de notre système repose sur un cycle en quatre étapes :

Collecte => Stockage => Analyse => Action

1. Exporters : Ils traduisent les données brutes (SNMP, métriques système) en un format lisible par Prometheus.
2. Prometheus : Il récupère ces données à intervalles réguliers et les stocke dans sa base de données temporelle.
3. Alertmanager & Python-Blocker : En cas de dépassement de seuil, une alerte est émise vers notre script Python pour déclencher une action de sécurité.
4. Grafana : Il interroge Prometheus pour générer des tableaux de bord visuels.

```
(brau@brau) ~/GIT/ENSEA/Projet_3A_Monitoring/monitoring
$ sudo docker-compose up -d
[sudo] password for brau:
Creating network for "monitoring" with IPAM driver "default"
Creating container python-blocker
Creating container node_exporter
Creating container snmp_exporter
Creating container alertmanager
Creating container prometheus
Creating container grafana
Container python-blocker started
Container node_exporter started
Container snmp_exporter started
Container alertmanager started
Container prometheus started
Container grafana started
```

Figure 1 : Démarrage de tous les services utilisés dans le cadre du projet

```
(brau@brau) ~/GIT/ENSEA/Projet_3A_Monitoring/monitoring
$ sudo docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2e73c0e9f6ad	grafana/grafana:latest	"/run.sh"	8 seconds ago	Up 6 seconds	0.0.0.0:3000→3000/tcp, :::3000→3000/tcp	grafana
515b0502dace	prom/prometheus:latest	"/bin/prometheus --c..."	8 seconds ago	Up 6 seconds	0.0.0.0:9090→9090/tcp, :::9090→9090/tcp	prometheus
78fc04bef0fb	monitoring-python-blocker	"python main.py"	8 seconds ago	Up 7 seconds	0.0.0.0:5000→5000/tcp, :::5000→5000/tcp	python-blocker
f1d5ab2c9f3	prom/alertmanager:latest	"/bin/alertmanager -..."	8 seconds ago	Up 7 seconds	0.0.0.0:9093→9093/tcp, :::9093→9093/tcp	alertmanager
91074a48554d	prom/node-exporter:latest	"/bin/node_exporter -..."	8 seconds ago	Up 7 seconds	0.0.0.0:9100→9100/tcp, :::9100→9100/tcp	node_exporter
bcefdfe3b23a	prom/snmp-exporter:latest	"/bin/snmp_exporter -..."	8 seconds ago	Up 7 seconds	0.0.0.0:9116→9116/tcp, :::9116→9116/tcp	snmp_exporter

Figure 2 : Liste de tous les conteneurs utilisés dans le cadre du projet

2.3. Mise en œuvre du Docker-Compose

La première étape du déploiement consiste à créer un fichier de configuration racine nommé *docker-compose.yml*. Ce fichier agit comme un chef d'orchestre : il définit et appelle les différents conteneurs nécessaires au projet.

Le format YAML a été choisi pour ce fichier pour trois raisons :

- **Lisibilité** : Sa structure par indentation rend l'architecture du réseau claire et facile à maintenir.

- **Portabilité** : Il permet de déployer l'intégralité du système sur n'importe quel système avec une commande unique.
- **Flexibilité** : On peut y déclarer précisément les dépendances entre services, les volumes de stockage et les réseaux isolés.

```

▶ Run All Services
services:
# ===== GRAFANA =====
▶ Run Service
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  volumes:
    - grafana:/var/lib/grafana
  depends_on:
    - prometheus

# ===== PROMETHEUS =====
▶ Run Service
prometheus:
  image: prom/prometheus:latest
  container_name: prometheus
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
    - ./alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
  ports:
    - "9090:9090"
  depends_on:
    - snmp_exporter
    - node_exporter
    - alertmanager

```

Figure 3 : Configuration du fichier docker-compose.yml

Paramètres	Utilité concrète
image	Définit le logiciel à installer (Grafana, Prometheus, etc.)
ports	Relie le conteneur au PC (ex: 3000 pour voir Grafana sur le navigateur)
volumes	Stockage : Empêche la perte de données et lie les fichiers de config au Docker
depends_on	Ordre : Force un service à attendre que les autres soient lancés avant de démarrer
container_name	Donne un nom simple au conteneur pour le gérer plus facilement en ligne de commande

3. Collecte et Traitement des Métriques

Cette phase est le socle du projet. Elle consiste à transformer des informations provenant d'équipements hétérogènes (serveurs de l'école et machine locale) en métriques exploitables par notre système de sécurité.

3.1. Architecture de collecte (Prometheus) : Le modèle "Pull"

Prometheus est le cerveau de notre système, il assure la collecte des données SNMP et leur stockage chronologique. Il transforme ces flux bruts en métriques exploitables pour permettre à Grafana de les visualiser et à l'Alertmanager de les analyser.

Contrairement aux systèmes classiques qui attendent de recevoir des données, Prometheus fonctionne sur un modèle "Pull". C'est lui qui va interroger les différents "Exporters" à intervalles réguliers (le scrape interval). Cette méthode garantit que le système de supervision ne soit pas submergé par un flux de données non contrôlé.

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

# ===== ALERTING (au bon endroit) =====
alerting:
  alertmanagers:
    - static_configs:
      - targets: ['alertmanager:9093']

# ===== ALERT RULES =====
rule_files:
  - "alert_rules.yml"

scrape_configs:

  # ===== PROMETHEUS =====
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  # ===== SNMP UBIQUITI =====
  - job_name: 'snmp-ubiquiti'
    metrics_path: /snmp

    params:
      module: [ubiquiti]
      auth: [public_v2]

    static_configs:
      - targets:
          - 10.100.0.3
          - 10.100.1.3

    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target

      - source_labels: [__param_target]
        target_label: instance

      - target_label: __address__
        replacement: snmp_exporter:9116
```

scrape_interval (15s) : C'est la fréquence de collecte. Toutes les 15 secondes, Prometheus interroge les cibles.

rule_files : Prometheus charge le fichier *alert_rules.yml*. C'est ici qu'on définit ce qu'est une "attaque" ou une "anomalie".

targets : Prometheus est configuré pour envoyer immédiatement ses conclusions à l'Alertmanager.

Figure 4 : Configuration du fichier *prometheus.yml*

Pour confirmer l'interopérabilité des services, nous utilisons l'interface de Prometheus (Status > Targets). Tous nos jobs (Prometheus, Node Exporter et SNMP Ubiquiti) affichent l'état "UP". Cela prouve que les conteneurs partagent avec succès le même sous-réseau Docker et que les flux de données entre les collecteurs et la base de données temporelle sont parfaitement opérationnels.

Job	Endpoint	Labels	Last scrape	State
node_exporter	http://node_exporter:9100/metrics	instance="node_exporter:9100" job="node_exporter"	45m 55.944s ago S: 69ms	UP
prometheus	http://prometheus:9090/metrics	instance="prometheus:9090" job="prometheus"	50m 2.042s ago S: 7ms	UP
snmp-ubiquiti	http://snmp_exporter:9116/snmp	instance="18.188.1.3" job="snmp-ubiquiti"	45m 56.185s ago S: 492ms	UP
	auth="public_v2" module="ubiquiti" target="18.188.1.3"			
	http://snmp_exporter:9116/snmp	instance="18.188.8.3" job="snmp-ubiquiti"	50m 7.217s ago S: 492ms	UP
	auth="public_v2" module="ubiquiti" target="18.188.8.3"			

Figure 5 : Interface Web Prometheus - Targets avec jobs "UP"

3.2. Supervision Distante via SNMP Exporter

Cette méthode permet de surveiller les équipements réseaux (Ubiquiti) sans agent local, en interrogeant directement le protocole SNMP (Simple Network Management Protocol).

SNMP Exporter

Prometheus Exporter for SNMP targets

Version: (version=0.29.0, branch=HEAD, revision=42f8f2a991516b39ed60ef67eb69263d817ecd52)

- [Config](#)
- [Metrics](#)

Target:

Auth:

Module:

Download a detailed report of resource usage (pprof format, from the Go runtime):

- [heap usage \(memory\)](#)
- [CPU usage \(60 second profile\)](#)

To visualize and share profiles you can upload to pprof.me

Figure 5 : Interface Web SNMP Exporter

A. Le rôle de l'Object Identifier (OID)

Chaque indicateur de performance (état d'un port, température, trafic) possède une adresse numérique hiérarchisée unique appelée OID. Par exemple, l'OID **1.3.6.1.2.1.2** donne accès aux statistiques des interfaces.

B. La passerelle SNMP Exporter

Le snmp_exporter sert de traducteur : il interroge les équipements via leurs OID complexes et convertit les réponses brutes en métriques nommées et exploitables par Prometheus (ex: transformation d'un compteur d'octets en la métrique *ifInOctets*).

C. Analyse du fichier de configuration *snmp.yml*

Ce fichier pilote l'intelligence de l'exporter et se structure en trois sections critiques :

- **Authentification** (*auths*) : Définit les paramètres d'accès. Nous utilisons la communauté public en version **SNMP v2c**, standard de communication pour les équipements de l'école.
- **Exploration** (*walk*) : Liste les branches de l'arbre MIB à parcourir pour extraire l'uptime système, les débits d'interfaces et les journaux d'erreurs réseau.
- **Structuration des métriques** :
 - Typage : Le fichier distingue les **Counters** (valeurs incrémentales comme le *sysUpTime*) des **Gauges** (valeurs fluctuantes comme l'état opérationnel *ifOperStatus*).
 - Indexation : L'utilisation des indexes (*ifIndex*) permet de lier chaque donnée brute à un label (nom d'interface). Dans Grafana, cela permet d'identifier précisément quel port du switch subit une saturation ou une déconnexion.

3.3. Supervision Locale via Node Exporter

En complément de la vue réseau, le Node Exporter est déployé pour surveiller les ressources de la machine de contrôle (PC de supervision).

- Objectif : Mesurer en temps réel les constantes matérielles critiques : charge CPU, utilisation de la mémoire RAM et entrées/sorties Disque.
- Utilité stratégique : Cette surveillance locale permet la corrélation de données. Elle permet de vérifier si une hausse critique du trafic réseau (détectée via SNMP) engendre une surcharge de traitement sur le poste de supervision, garantissant ainsi la fiabilité de nos propres outils.

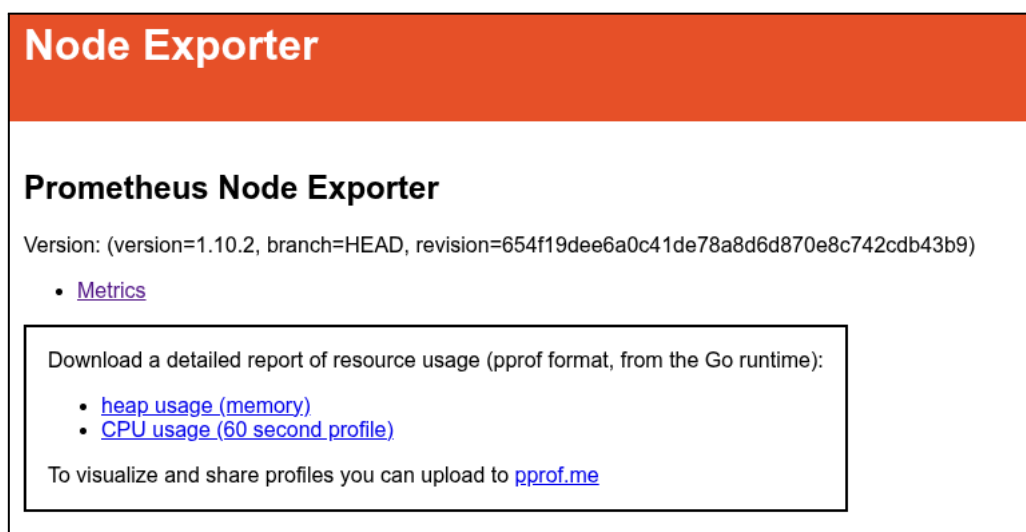


Figure 6 : Interface Web Node Exporter

4. Visualisation et Système d'Alerte

Une fois la collecte opérationnelle, l'enjeu est de transformer ces milliers de lignes de données en informations exploitables pour un administrateur réseau.

4.1. Grafana : Dashboarding et Observabilité

Grafana est l'interface visuelle de notre système. En connectant Prometheus comme source de données (Data Source), nous avons conçu des tableaux de bord dynamiques :

- **Monitoring Réseau** : Visualisation en temps réel du trafic entrant et sortant des équipements Ubiquiti (en bits/s).
- **Santé Système** : Graphiques d'utilisation CPU et mémoire de la machine locale pour surveiller l'impact de la supervision sur les ressources.
- **Corrélation** : L'interface permet de superposer les courbes pour identifier si un pic de trafic correspond à une saturation CPU.

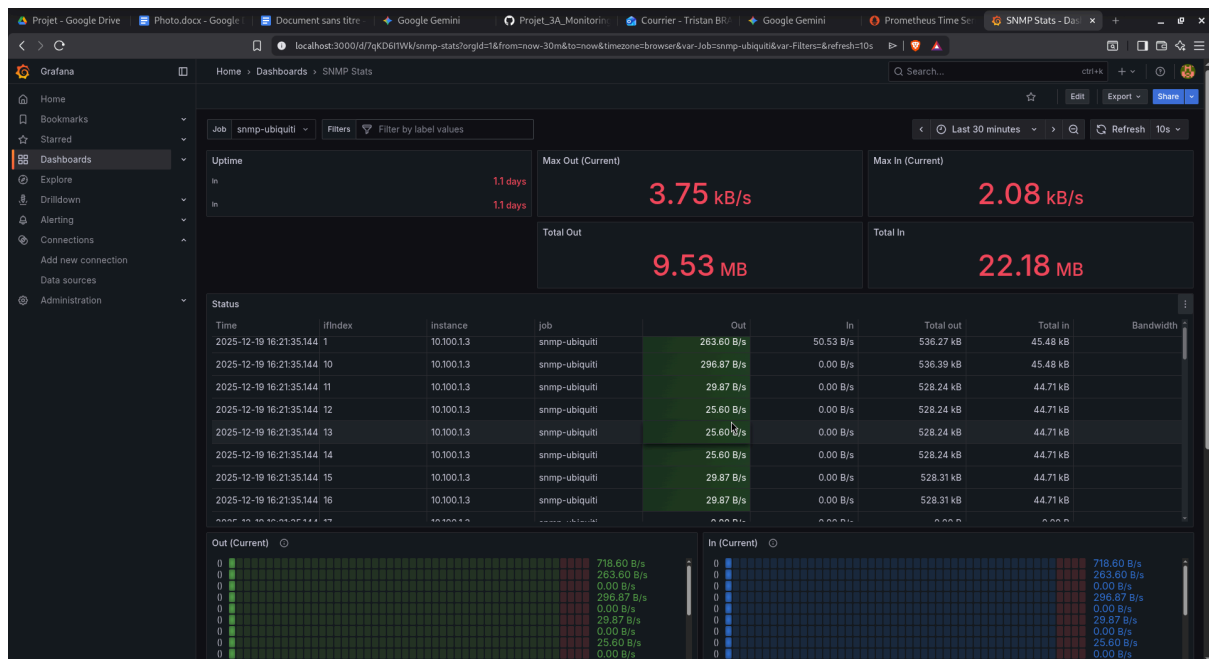


Figure 7.1 : Interface Web Grafana - Dashboard Métriques

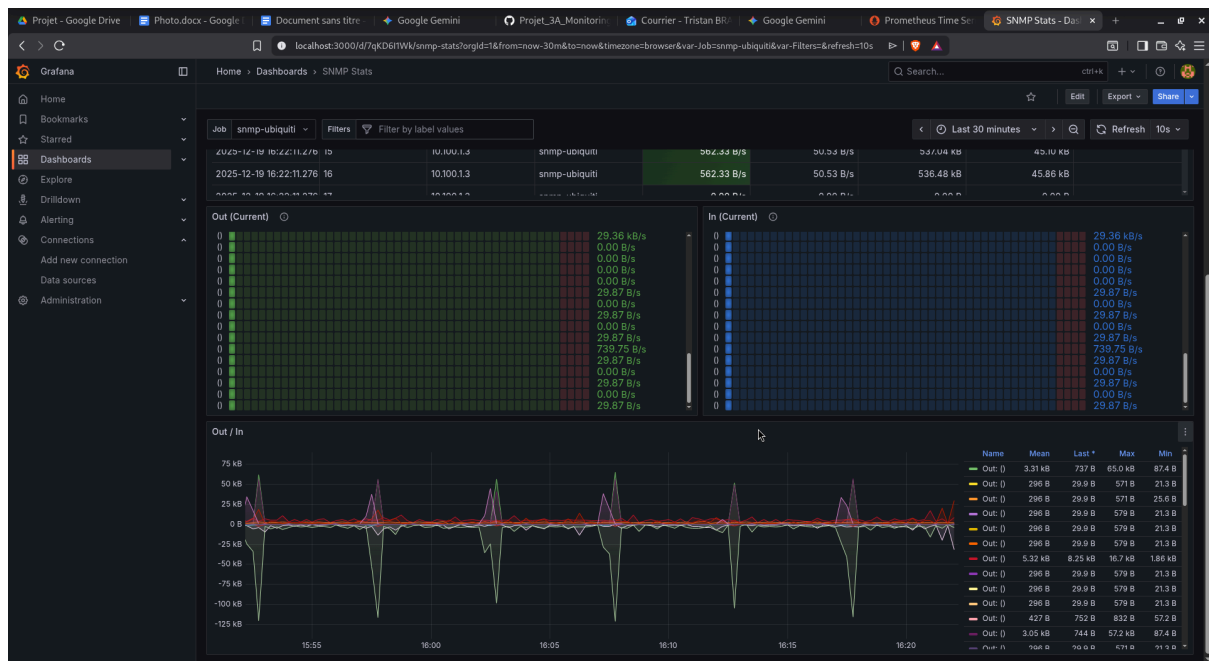


Figure 7.2 : Interface Web Grafana - Dashboard Courbes

4.2. Alertmanager : La logique de détection

L'Alertmanager est le composant critique qui fait le lien entre la surveillance et l'action. Il fonctionne selon un flux précis défini dans notre fichier `alert_rules.yml`.

1. Définition des règles : Nous avons configuré des seuils critiques (ex: Trafic > 90% de la capacité pendant plus de 2 minutes).
2. États de l'alerte :
 - Pending : Le seuil est franchi, mais le système attend de voir si cela dure pour éviter les faux positifs.
 - Firing : L'anomalie est confirmée, l'alerte est envoyée.
3. Routage vers le Python-Blocker : Contrairement à une alerte classique par mail, notre Alertmanager envoie un Webhook (une requête HTTP POST) directement à notre conteneur Python-Blocker avec les détails de l'incident (IP, Port, type d'attaque).

4.3. Notification des alertes

Pour garantir qu'aucune anomalie ne passe inaperçue, nous avons configuré l'Alertmanager comme une tour de contrôle des notifications. Son rôle est de recevoir les alertes brutes de Prometheus et de les distribuer intelligemment.

Configuration du fichier `alertmanager.yml` : Ce fichier définit les "receveurs"

Nous avons mis en place une double notification :

- Notification Mail (SMTP) : Une alerte est envoyée automatiquement sur nos boîtes mails dès qu'un seuil critique est atteint sur les serveurs Ubiquiti.
- Notification Webhook (Automatique) : L'alerte est simultanément transmise à notre conteneur Python-Blocker pour déclencher le blocage de port.

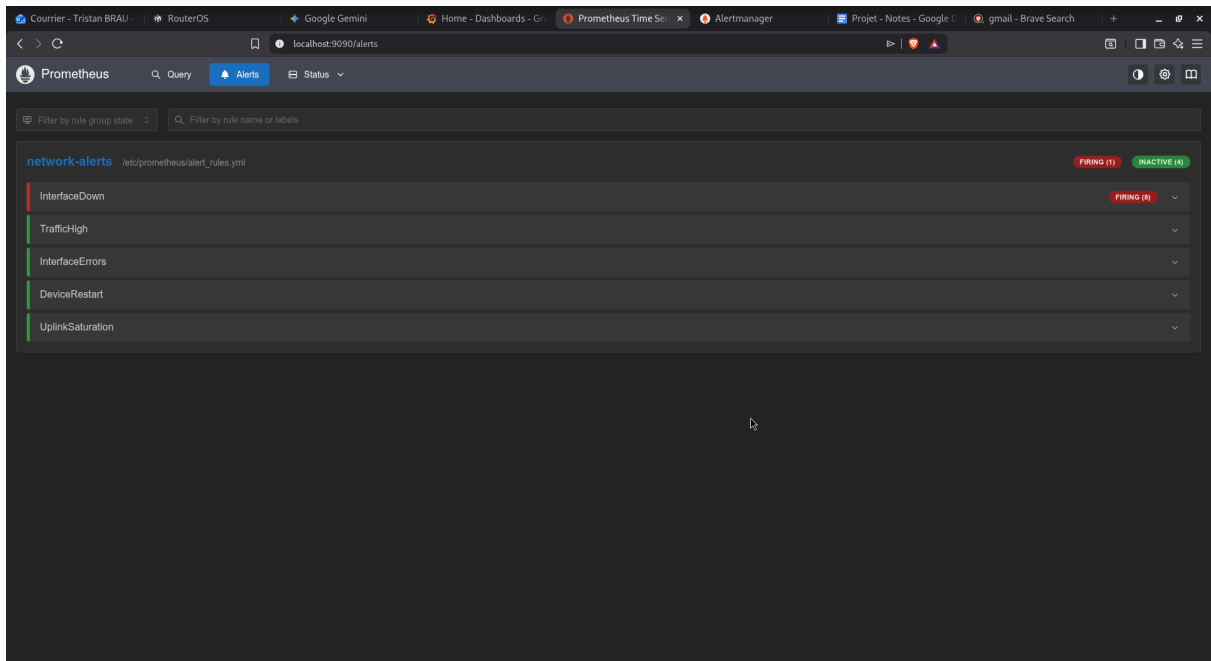


Figure 8 : Interface Web Prometheus - Alerts

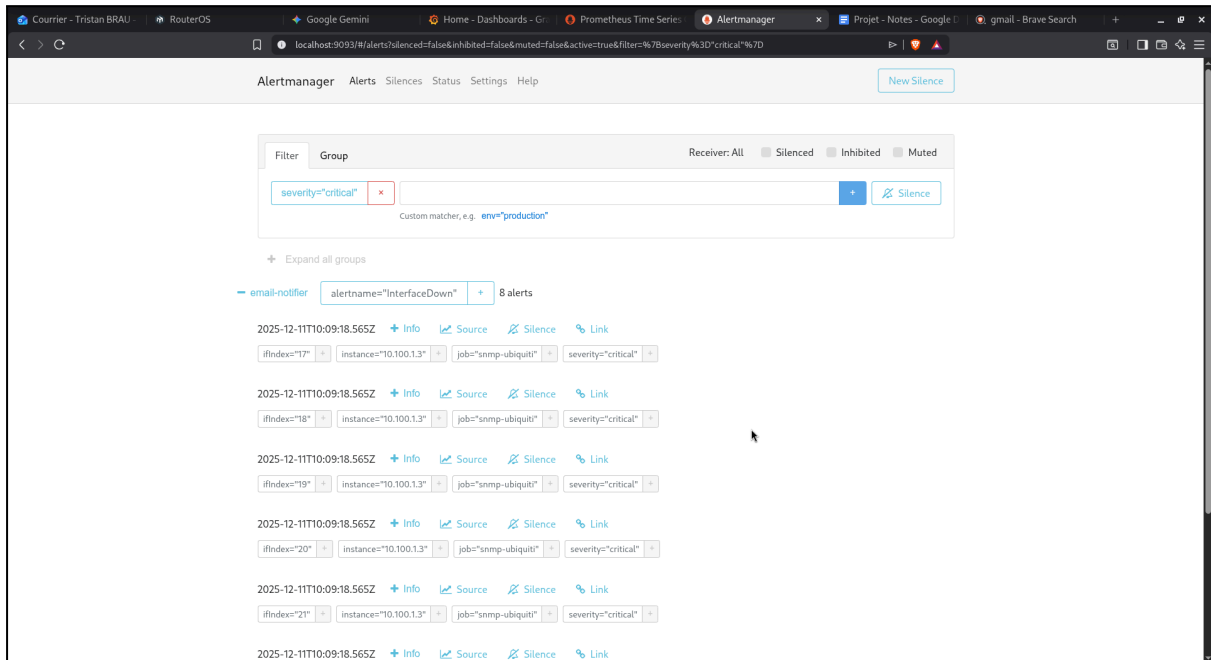


Figure 9 : Interface Web Alertmanager - Visualisation des alertes

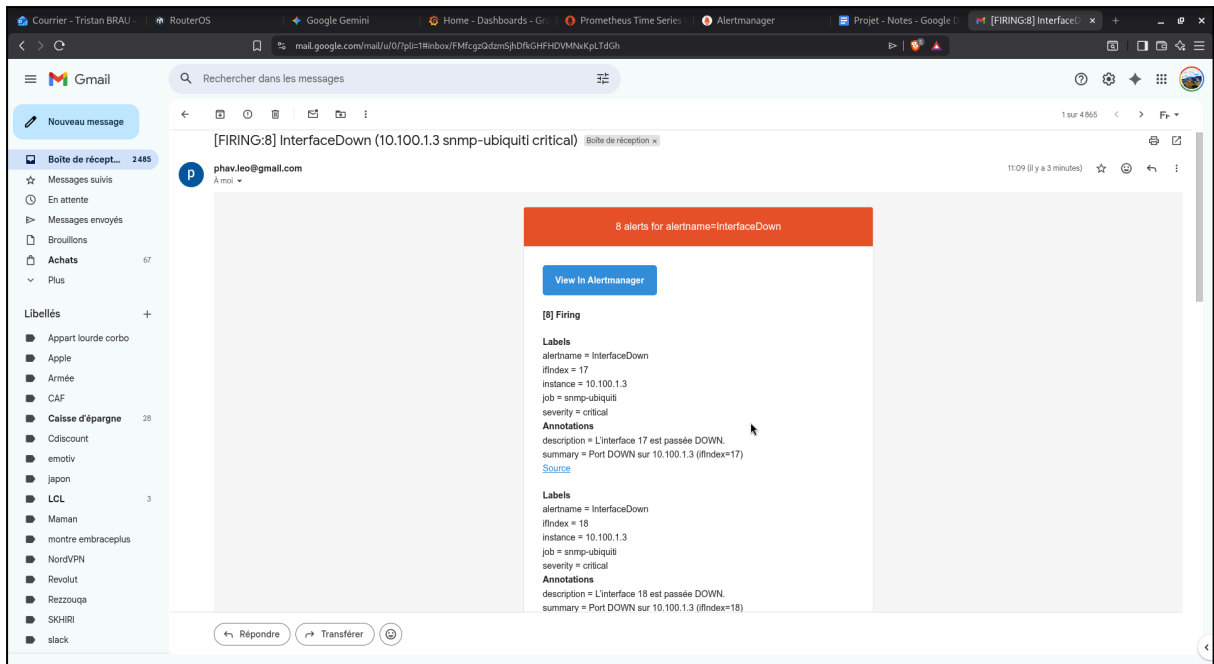


Figure 10 : Notification de l'alerte par mail (SMTP)

5. Sécurité et Automatisation : Le module Python-Blocker

Le composant Python-Blocker est le bras armé de notre architecture. Il agit comme un IPS (Intrusion Prevention System) capable d'intervenir en temps réel sur l'infrastructure sans intervention humaine.

5.1. Architecture du module : Flask comme centre de commande

Contrairement aux autres composants du système qui sont des solutions "sur étagère", ce module est un développement sur mesure. Nous avons choisi le framework Flask pour transformer notre script de sécurité en un micro-service réactif intégré dans un conteneur Docker.

- Le serveur d'écoute Flask : Le script fait tourner un serveur Web léger qui expose une route API spécifique (un Webhook). Ce point d'entrée est maintenu en écoute permanente pour recevoir les requêtes POST envoyées par l'Alertmanager.
- Analyse et Parsing JSON : Dès qu'une alerte "High Traffic" est détectée, Flask reçoit un objet JSON structuré. Le script extrait alors instantanément les informations critiques :
 - L'instance concernée (IP du serveur ou équipement Ubiquiti).
 - Le port cible ayant déclenché l'alerte.
 - Le statut de l'alerte (pour ne déclencher le blocage que si l'alerte est confirmée en mode firing).

Cette architecture permet de séparer la logique de détection (Prometheus) de la logique d'action (Flask), rendant le système à la fois modulaire et extrêmement rapide.

5.2. Sécurisation des échanges

La sécurité étant au cœur de notre projet ENSEA, nous avons implémenté une vérification par Token pour protéger notre API Flask :

- Le mécanisme : L'Alertmanager doit envoyer un header HTTP nommé X-Webhook-Token. Si ce token est absent ou incorrect, le script refuse l'exécution (Erreur 403). Cela évite qu'un attaquant puisse fermer des ports arbitrairement en appelant l'API.

5.3. Protocole de Test et Résultats

Pour tester notre stack, nous avons simulé une saturation de trafic sur l'un des ports du routeur.

1. Détection : Prometheus a identifié le dépassement de seuil grâce aux métriques SNMP.
2. Transmission : L'Alertmanager a envoyé un Webhook JSON vers notre serveur Flask.
3. Réaction : Le script a immédiatement ouvert une session SSH via la bibliothèque Paramiko pour désactiver l'interface réseau concernée sur le MikroTik.



```
(brau@Brau)-[~]
$ curl -X POST \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Token: 1234" \
  -d '{
    "alerts": [
      {
        "status": "firing",
        "labels": {
          "alertname": "TrafficHigh",
          "ifIndex": "2"
        }
      }
    ]
  }' http://localhost:5000/webhook
Accès refusé

(brau@Brau)-[~]
$ curl -X POST \
  -H "Content-Type: application/json" \
  -H "X-Webhook-Token: 123" \
  -d '{
    "alerts": [
      {
        "status": "firing",
        "labels": {
          "alertname": "TrafficHigh",
          "ifIndex": "2"
        }
      }
    ]
  }' http://localhost:5000/webhook
{"status": "success"}
```

Figure 11 : Simulation d'une alerte par Webhook

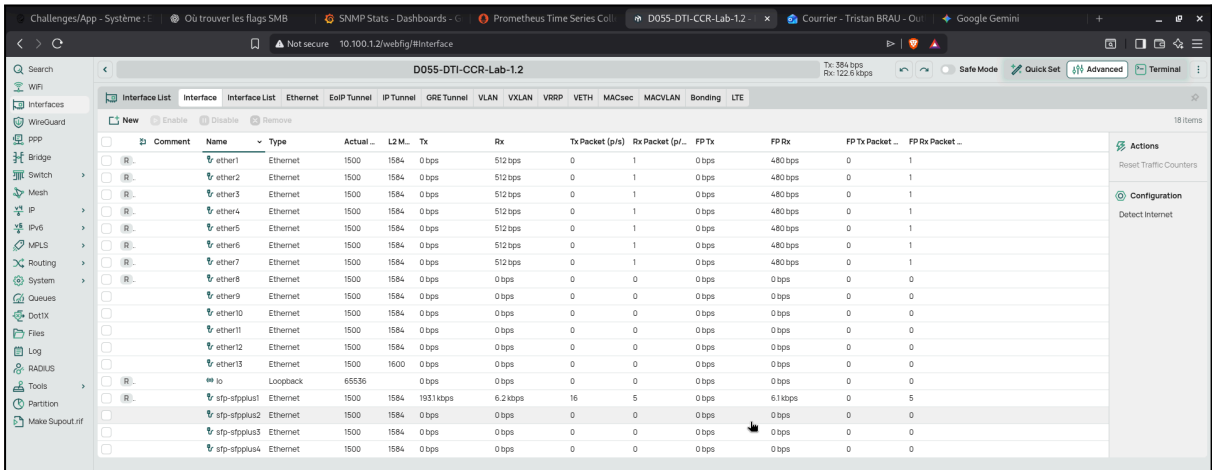


```
(brau@Brau)-[~/GIT/ENSEA/Projet_3A_Monitoring/monitoring]
$ sudo docker logs -f python-blocker
* Serving Flask app 'main'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.19.0.5:5000
Press CTRL+C to quit
X Accès refusé : Mauvais Token
172.19.0.1 - - [17/Dec/2025 16:16:03] "POST /webhook HTTP/1.1" 403 -
Données reçues : {'alerts': [{'status': 'firing', 'labels': {'alertname': 'TrafficHigh', 'ifIndex': '2'}}]}
Analyse : TrafficHigh (firing) sur Port 2
Commande SSH exécutée sur MikroTik : /interface disable [find default-name="ether2"]
172.19.0.1 - - [17/Dec/2025 16:16:19] "POST /webhook HTTP/1.1" 200 -
Données reçues : {'status': 'resolved', 'alerts': [{'status': 'resolved', 'labels': {'alertname': 'TrafficHigh', 'ifIndex': '1'}}]}
Analyse : TrafficHigh (resolved) sur Port 1
Commande SSH exécutée sur MikroTik : /interface enable [find default-name="ether1"]
172.19.0.1 - - [17/Dec/2025 16:26:44] "POST /webhook HTTP/1.1" 200 -

```

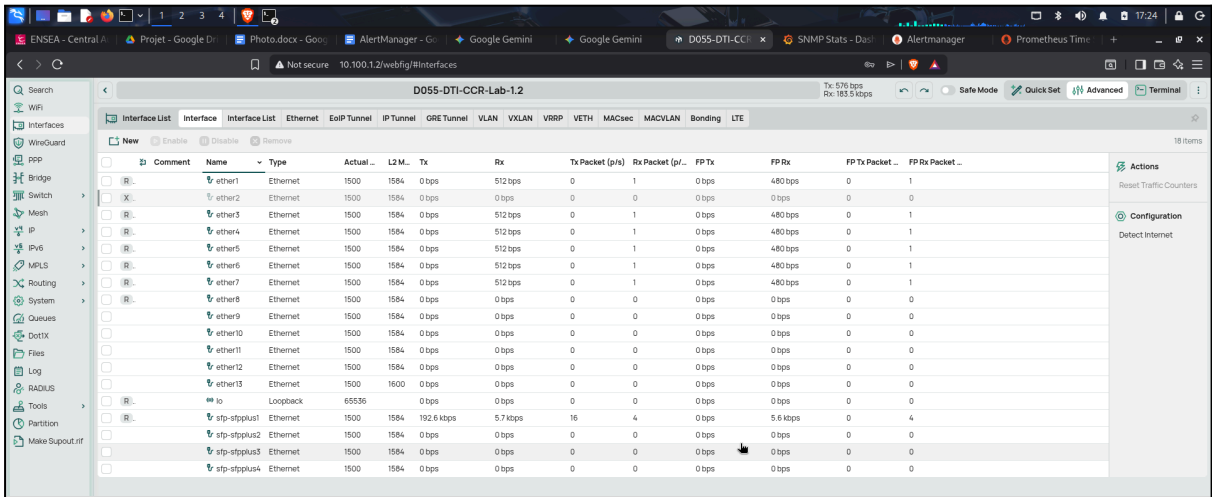
Figure 12 : Logs du service Python-Blocker

Validation sur l'équipement : L'efficacité de la remédiation est confirmée directement dans l'interface de gestion du MikroTik. Nous constatons que l'état de l'interface réseau est passé instantanément en mode "disabled" suite à l'exécution de la commande SSH par le script, prouvant ainsi la réussite de l'automatisation.



	Comment	Name	Type	Actual ...	L2 M...	Tx	Rx	Tx Packet (p/s)	Rx Packet (p/s)	FP Tx	FP Rx	FP Tx Packet ...	FP Rx Packet ...
☐	R.L.	ether1	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☒	R.L.	ether2	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether3	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether4	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether5	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether6	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether7	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether8	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether9	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether10	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether11	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether12	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether13	Ethernet	1500	1600	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	lo	Loopback	65536	0 bps	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	stp-stpplus1	Ethernet	1500	1584	193.1 kbps	6.2 kbps	16	5	0 bps	6.1 kbps	0	5
☐	R.L.	stp-stpplus2	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	stp-stpplus3	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	stp-stpplus4	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0

Figure 13.1 : Interface de gestion Web MikroTik



	Comment	Name	Type	Actual ...	L2 M...	Tx	Rx	Tx Packet (p/s)	Rx Packet (p/s)	FP Tx	FP Rx	FP Tx Packet ...	FP Rx Packet ...
☐	R.L.	ether1	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☒	X.L.	ether2	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether3	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether4	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether5	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether6	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether7	Ethernet	1500	1584	0 bps	512 bps	0	1	0 bps	480 bps	0	1
☐	R.L.	ether8	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether9	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether10	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether11	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether12	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	ether13	Ethernet	1500	1600	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	lo	Loopback	65536	0 bps	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	stp-stpplus1	Ethernet	1500	1584	192.6 kbps	5.7 kbps	16	4	0 bps	5.6 kbps	0	4
☐	R.L.	stp-stpplus2	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	stp-stpplus3	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0
☐	R.L.	stp-stpplus4	Ethernet	1500	1584	0 bps	0 bps	0	0	0 bps	0 bps	0	0

Figure 13.2 : Interface de gestion Web MikroTik - Interface ether2 désactivée

6. Guide de Déploiement

Cette section détaille les étapes nécessaires pour reproduire et lancer l'intégralité de l'architecture micro-services sur une nouvelle machine Linux.

6.1. Prérequis et Lancement Initial

Avant de démarrer, s'assurer que **Docker** et **Docker-Compose** sont installés sur le système. Le déploiement s'effectue en une commande unique depuis la racine du projet :

Démarrage des services : `sudo docker-compose up --build -d`

L'option `--build` est cruciale ici car elle force la construction locale de l'image du conteneur **Python-Blocker** à partir de notre code source. L'option `-d` lance l'infrastructure en arrière-plan.

6.2. Vérification de l'état des conteneurs

Une fois le lancement effectué, il est nécessaire de valider que tous les processus sont opérationnels.

- **Contrôle via le terminal :** `sudo docker ps -a`

Cette commande permet de vérifier que les six services (grafana, prometheus, python-blocker, alertmanager, node_exporter, snmp_exporter) affichent le statut "Up".

- **Validation des flux de données :** Accédez à l'interface de Prometheus (<http://localhost:9090/targets>) pour confirmer que les collecteurs parviennent à interroger les équipements cibles.

6.3. Monitoring en temps réel des actions de défense

Pour observer le comportement du système lors d'une alerte et valider le bon fonctionnement du module de remédiation, utilisez la commande suivante :

`sudo docker logs -f python-blocker`

Cette commande affiche en continu les journaux du script Flask. Elle permet de visualiser l'arrivée des Webhooks, la validation du Token de sécurité et l'exécution des commandes SSH vers le MikroTik.

7. Conclusion

Le projet de supervision active réalisé à l'ENSEA nous a permis de concevoir une architecture logicielle robuste et sécurisée. Nous avons dépassé le simple stade de l'observabilité passive pour mettre en œuvre un véritable **système de défense actif**.

Synthèse des acquis techniques :

- **Maîtrise de la donnée** : L'utilisation de Prometheus et du modèle "Pull" garantit une collecte efficace et stable des métriques système et réseau (SNMP).
- **Observabilité avancée** : Grafana offre une visibilité totale, permettant de corréler l'état matériel de notre poste de supervision avec le trafic réel des équipements Ubiquiti.
- **Cybersécurité proactive** : L'implémentation du module **Python-Blocker** démontre qu'une architecture micro-services peut réagir intelligemment et de manière autonome face à un incident, en isolant dynamiquement les ports réseau compromis.

En conclusion, cette solution allie la flexibilité de la conteneurisation à la puissance de l'automatisation. Elle répond aux enjeux critiques de continuité de service et de sécurité des infrastructures modernes. Ce projet valide notre capacité à intégrer des domaines variés (réseaux, administration système et développement logiciel) pour créer une solution d'ingénierie complète.